Correction exercice TD3

CPES « Sciences des données, arts et cultures »

Antoine GAUQUIER — antoine.gauquier@ens.fr
Pierre ABOULKER — pierreaboulker@gmail.com

04 novembre 2025

Question 1 Donner le pseudo-code d'un algorithme *naïf* (ne faisant pas appel à des méthodes de recherche avancées) qui résoud ce problème. Cet algorithme prend donc en entrée un tableau de nombres triés par ordre croissant A, ainsi qu'un nombre x. Il retournera « Vrai » si x est *majoritaire* dans A, et « False » sinon.

La façon la plus « naturelle » est de parcourir le tableau linéairement, et de compter les occurrences de x. Il suffit ensuite de comparer ce nombre avec $\frac{n}{2}$ et de conclure.

Algorithme 1 EST_MAJORITAIRE_NAIF

Entrée: A le tableau trié par ordre croissant, x l'élément dont on veut déterminer s'il est majoritaire.
Sortie: Vrai si x est majoritaire dans A, faux sinon.

```
1: n←|A|   2
2: cpt ←0   1
3: pour i allant de 1 à n faire   3n : i ⩽ n et i ← i + 1, le tout n fois
4:     si A[i]=x alors   2n
5:         cpt ← cpt + 1   2n
6:     fin si
7: fin pour
8: si cpt > n/2 alors   2
9:     Retourner Vrai   1 : on compte soit celui-ci, soit le suivant (même au pire cas, on ne retourne qu'une seule fois)
10: fin si
11: Retourner Faux
```

Question 2 Donner l'expression $f(n)$ de la fonction f qui décrit le nombre d'opérations élémentaires *au pire cas* réalisé par cet algorithme en fonction de n. En déduire sa complexité asymptotique en O.

Pour déterminer $f(n)$, il faut définir quelles sont les opérations élémentaires que l'on considère. Ici, on comptera les affectations, calculs de taille de liste, opérations arithmétiques, comparaisons et retours. Dans l'Algorithme 1, elles sont écrites en orange et comptées en fin de ligne. Au total, on a donc $f(n) = 2 + 1 + 3n + 4n + 2 + 1 = 7n + 6 \in O(n)$.

Question 3 Donner le pseudo-code d'un meilleur algorithme que celui proposé en question 1, dont la complexité asymptotique est meilleure. Cet algorithme devra être *récursif*. *Indice : Vous pouvez vous inspirer par exemple de la recherche dichotomique...*

L'idée consiste à exploiter le fait que les nombres sont triés par ordre croissant dans A. En effet, pour déterminer si x est majoritaire, il suffit de localiser la première occurrence de x dans A (si elle existe), et regarder si l'élément situé $\frac{n}{2}$ positions plus loin vaut toujours x ou non. Si c'est le cas, alors x est majoritaire, sinon, il ne l'est pas.

On va utiliser le principe de la recherche dichotomique pour localiser la première occurrence de x. Écrivons d'abord le pseudo-code de la fonction principale : il est donné dans l'Algorithme 2.

Algorithme 2 EST_MAJORITAIRE

Entrée: A le tableau trié par ordre croissant, x l'élément dont on veut déterminer s'il est majoritaire.
Sortie: Vrai si x est majoritaire dans A, faux sinon.

```
1: n ← |A|                                   2
2: debut ← RECH_PREM_OCC(A, x, 1, n)         1 + T(n)
3: si debut = -1 alors                       1
4:     Retourner Faux                        1 : on compte le return ici pour tous les autres
5: fin si
       fin_minimum ← debut + ⌊n/2⌋            4
6: si fin_minimum ⩽ n et A[fin_minimum] = x alors   3
7:     Retourner Vrai
8: fin si
9: Retourner Faux
```

Puis la fonction qui recherche par dichotomie la position de la première occurrence de x (et retourne -1 si x n'est pas dans A) : il est donné dans l'Algorithme 3.

Question 4 Donner la relation de récurrence $T(n)$ de cet algorithme. En utilisant la méthode de votre choix, donner la complexité asymptotique en Θ de ce dernier. Conclure.

Analysons la complexité de l'Algorithme 2 : les opérations élémentaires y sont notées en orange. On note $T(n)$ la complexité de la fonction décrite par l'Algorithme 3. On obtient un nombre d'opérations élémentaires pour l'Algorithme 2 $f(n) = T(n) + 12$: autrement dit, la complexité asymptotique en O de l'Algorithme 2 est la même que celle de l'Algorithme 3.

Analysons donc celle de l'Algorithme 3 : on a la relation de récurrence $T(n) = T\left(\frac{n}{2}\right) + 17 = T\left(\frac{n}{2}\right) + g(n)$ où $g(n) \in O(1)$. Nous avons déjà résolu cette formule de récurrence dans le TD/TP2 (c'est la même que celle de la recherche dichotomique).

On sait alors que l'on a $T(n) \in \Theta(\log(n))$, qui donne $f(n) \in \Theta(\log(n))$. Cette version a bien une meilleure complexité asymptotique que celle de la première question, qui était linéaire.

Algorithme 3 RECH_PREM_OCC

Entrée: A le tableau trié par ordre croissant, x l'élément dont on veut déterminer s'il est majoritaire, `gauche` et `droite` les positions dans lesquelles effectuer la recherche dans A.

Sortie: La position de la première occurrence de x dans $A[\texttt{gauche} : \texttt{droite}]$, ou -1 si $x \notin A$.

1: **si** `gauche` $>$ `droite` **alors** **1**
2: **Retourner** -1 **1** : on compte le return ici
3: **fin si**
`milieu` $\leftarrow \left\lfloor \frac{\texttt{gauche}+\texttt{droite}}{2} \right\rfloor$ **4**
4: **si** $A[\texttt{milieu}] = x$ **et** (`milieu` $= 0$ **ou** $A[\texttt{milieu}-1] \neq x$) **alors** **8**
5: **Retourner** `milieu`
6: **sinon si** $A[\texttt{milieu}] \geqslant x$ **alors** **3**
7: **Retourner** RECH_PREM_OCC(A, x, `gauche`, `milieu` - 1) $T\left(\frac{n}{2}\right)$: le suivant n'est pas compté, car c'est soit le cas du retour de `milieu`, soit ce retour, soit le suivant, ce qui au pire cas est la complexité d'un des deux appels récursifs
8: **sinon**
9: **Retourner** RECH_PREM_OCC(A, x, `milieu`, `droite`)
10: **fin si**
11: **Retourner** Faux